Finding heart disease in an imbalanced survey

What feature encoding and decision thresholds change

Ayhan Suleymanzade · Jasna Ilieva · Hayden Richard Stec

EPFL CS-433 · Fall 2026

Abstract. Only 8.83% of respondents in this dataset report heart disease. Predicting the negative class for everyone therefore gives 91.17% accuracy while finding no positive cases. We study how simple NumPy models handle this imbalance, separating model fitting, selection, and evaluation. Our selected logistic model reaches a holdout F1 of **0.4214**. Categorical encoding and squared features give a modest gain. The larger change comes from the decision threshold: lowering it from 0.5 to 0.21065 raises F1 from 0.2540 to 0.4214 by recovering more positive cases, at the cost of more false alarms.

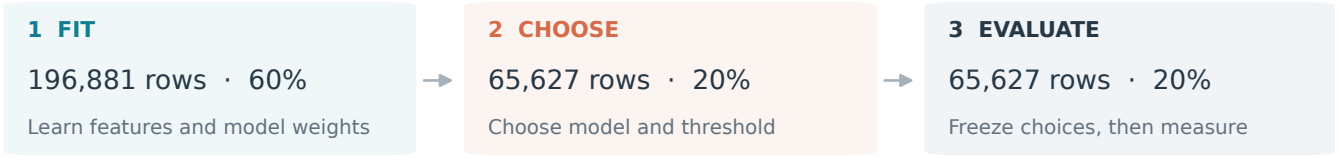


Figure 1: Freeze model choices before evaluating the holdout. The split preserves class proportions. Preprocessing is learned on training rows; model and threshold selection use tuning rows. The holdout is opened only after those choices are frozen.

1 Why accuracy is not enough

The task is to identify respondents who report a previous heart attack or coronary heart disease from their survey answers. The course data contain 328,135 labeled respondents, 321 input variables, and 109,379 unlabeled test respondents [1]. The label records disease history at the time of the survey [2].

With only about nine positive cases in every hundred respondents, a model can achieve high accuracy simply by predicting the majority class. We therefore select models using F1, which balances *precision* (how many flagged cases are positive) and *recall* (how many positive cases are found). We also report accuracy and the area under the ROC curve (AUROC), which measures how well the scores rank positives above negatives. The leaderboard displays F1 first, while its rules describe accuracy [3]; reporting both makes the tradeoff explicit.

2 Preparing the features

We first remove record identifiers and information that can reveal the answer directly. In particular, `HAREHAB1` asks about rehabilitation after a heart attack. Its skip rule depends on a diagnosis used to define the target, so even whether the question was answered can disclose the outcome [2].

For the remaining variables, we remove columns with more than 95% missing entries and drop constant columns, preserving a missingness indicator when it varies. Other retained columns also get a missingness indicator when needed. We treat nonfinite entries as missing; codes such as 7 or 99 keep their original values because their meaning depends on the survey question.

Our **numeric representation** fills missing values with training medians, clips at the training 0.5th and 99.5th percentiles, then centers and scales each column. Our **engineered representation** gives categorical answers separate indicators and adds standardized squared continuous features, allowing a variable to have a curved relationship with the prediction. Columns with at most 20 observed values are categorical. Categories seen fewer than 20 times share an "other" indicator with unseen values; the most frequent category is the reference. The two representations contain 414 and 1136 development features, including an intercept.

3 Fitting and choosing the model

Figure 1 shows the evaluation design: a stratified 60/20/20 split with seed 433. Every preprocessing statistic comes from training rows. We compare least squares and ridge regression ($\lambda = 10^{-3}$) on numeric features, then logistic regression on both representations with $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}\}$. Logistic regression minimizes mean binary cross-entropy plus $\lambda\|w_{1:}\|_2^2$, leaving the intercept unpenalized. Our NumPy L-BFGS solver uses ten history pairs, Armijo backtracking, a gradient tolerance of 10^{-6}, and at most 150 iterations. We use no class weights or resampling.

For each candidate, we choose the score threshold that maximizes tuning F1, treating tied scores together. We then freeze the candidate with the highest tuning F1 before computing any holdout result. This keeps model selection separate from evaluation. The experiment uses one split; all holdout comparisons below describe these fixed choices.

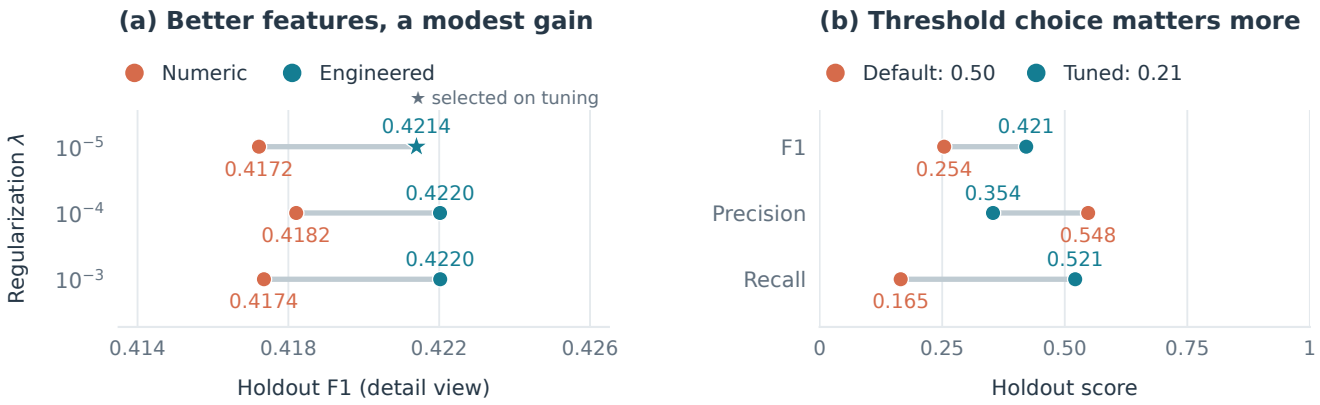


Figure 2: Feature encoding helps modestly; the threshold changes which cases are found. Left: numeric and engineered logistic models at the same regularization strengths, each with its own tuning-selected threshold. Right: the selected model at two thresholds, with identical fitted weights and scores. All values are measured on the holdout. The selected configuration is determined by tuning F1.

4 What changes the predictions?

The tuning set selects engineered logistic regression with $\lambda = 10^{-5}$ and threshold 0.21065. On the holdout, it reaches F1 0.4214, accuracy 0.8736, and AUROC 0.8578. Table 1 places this result alongside the simpler models.

Model	Tune F1	Holdout F1	Accuracy
Always negative	–	0.0000	0.9117
Least squares	0.4213	0.4133	0.8621
Ridge	0.4201	0.4145	0.8603
Selected logistic	0.4313	0.4214	0.8736

Table 1: Simple baselines provide useful context. Each fitted model uses its tuning-selected threshold. Accuracy is measured on the holdout; the full eight-candidate search is saved with the code.

Feature encoding gives a small gain. At the same regularization strength, $\lambda = 10^{-5}$, engineered features raise holdout F1 from 0.4172 to 0.4214 (Figure 2, left). Tuning regularization separately for each representation gives a similar comparison: 0.4174 versus 0.4214. This experiment changes categorical encoding and squared features together, so it cannot separate their contributions. The small difference also needs repeated splits before we can judge how consistently it holds.

The threshold makes the larger difference. A logistic model assigns each respondent a score; the threshold decides which scores become positive predictions. At 0.5, the selected model finds only 16.5% of positive respondents. Lowering it to 0.21065, as chosen on the tuning set, raises recall to 52.1%. More respondents are flagged, so precision falls from 54.8% to 35.4%. F1 rises from 0.2540 to 0.4214, while accuracy falls from 91.42% to 87.36%. The scores and their ranking stay the same (Figure 2, right).

The selected decisions yield 3,021 true positives, 5,522 false positives, 2,774 missed positives, and 54,310 true negatives. A 95% bootstrap interval for F1 is [0.4113, 0.4319], using 2,000 multinomial resamples of these counts with seed 434. This interval captures holdout sampling for the fixed model; training, model selection, and the survey design add uncertainty beyond it.

5 Reproducing the result

After evaluation, we refit the chosen preprocessing and model on all 328,135 labeled rows, keeping the tuning-selected threshold. The final feature map has 1156 columns. The reported holdout scores describe the development model; the competition score of the refit is not yet known.

The repository includes all six required methods in `implementations.py`. Each returns final weights and unpenalized loss, with batch-size-one SGD and normal-equation least squares and ridge. `train.py` reproduces the split, search, evaluation, and refit. `run.py` loads the saved preprocessing and weights to generate the test CSV. Data hashes, metrics, and optimizer traces are recorded; the report and figures are rebuilt from those metrics. Modeling uses NumPy 2.5.4 only.

This study has one split and a small search. All logistic fits reach the 150-iteration budget, so we do not claim exact convergence. Survey codes need more question-specific treatment, and self-reporting and the sampling design limit generalization. Within this experiment, feature encoding improves holdout F1 slightly, while choosing a threshold changes how many positive cases the same model recovers.

References

[1] EPFL CS-433. *Project 1 description and dataset*, Fall 2026.

[2] CDC. *2015 BRFSS Codebook*, pp. 73, 106.

[3] AIcrowd. *EPFL Machine Learning Project 1: leaderboard* and rules. Accessed 26 October 2026.